Part 1: How We Track Topics Over Time (Methodology & Justification)

---

1. How the System Works (The Methodology)

To track how the topics change from day to day, we just can't look at the topic titles, because the Narrative Engine (NE) might rename them. Instead, I built a two-layered tracking systems:

- **Structural Tracking (The Ground Truth):** I used the *unique id* hashes of the post as the primary bridge between the runs, because topic identification numbers are randomly reassigned in each run. I calculated the Jaccard Similarity (Intersection over union) of the post sets between every valid topic in between runs. This provides mathematical proof of data continuity.
- **Semantic & Entity Tracking (The Context):** I extracted the *key_entities* to track specific information added or removed between matched topics to measure how the narrative has changed. Additionally, I generated a TF-IDF matrix for the combined *title* and *summary* text of the topics to calculate cosine similarity.
- **Lifecycle Classification (Handling Edge Cases):** I defined mathematical threshold to classify the edge cases by combining these metrics:
  - **Same/Continues:** High Jaccard post overlap and high TF-IDF similarity.
  - **Merged/Split:** If 90% of Topic A's posts map to Topic B, but they only make up 20% of Topic B's total volume, Topic A was mathematically **Merged** into a broader narrative. The inverse ratio identifies a **Split**.
  - **Poor Performing:** Topic with exceptionally low post counts, empty entity lists, or high overlap with the *-1* outlier group were flagged and assigned a low "quality score" to prevent them from creating false merges.

2. Justifications: Why I chose This Method

- **Strict Adherence to Constraints (Cost Efficiency):** The assignment clearly prohibited the use of LLM to compare topics to approximate cost restrictions. TF-IDF and set based Jaccard calculations are exceptionally computationally cheap, high vectorizable, and scale effortlessly to millions to topics without the latency or API costs of deep learning models.
- **Overcoming the "Title Trap":** relying solely on standard NLP string matching is brittle, the Narrative Engine might completely re-title a topic even if the underlying conversation is identical. Anchoring the pipeline on the immutable *post_id* guarantee that we track the actual data distribution first, and evaluate the metadata second.
- **Explainability:** Unlike the black box neural networks, this pipeline is entirely transparent. If a stakeholder asks why a topic was classified as a "Split", the pipeline can point to the exact directional ratio and the specific posts that diverged.

3. Validation Strategy: Proving the system works

I would implement the following validation framework, to prove this method is correct:

- **Golden Dataset (manual Annotation):** I would randomly sample 100 topic pairs across the three runs. A human annotator would manually review the summaries and post contents, labelling the relationships (e.g., Same, Merged, Split, Unrelated). I would then run my pipeline against this subset and calculate the **Precision, Recall, and F1-Score** of my pipelines automated classifications against the human ground truth.
- **Threshold Sensitivity Analysis:** The thresholds for identifying splits/merges (e.g., Jaccard > 0.5) are currently heuristic. I would run a sensitivity analysis, systematically tweaking these thresholds by 10% up or down to observe the distribution of classifications. If a 5% change in a threshold drastically alters 40% of the labels, the model is too brittle and requires tuning.
- **Sanity Checks (Conservation of Posts):** I would write automated unit tests to ensure no posts are "lost" in the logic. If Topic A splits into B and C, then union of posts in B and C should closely approximate the original volume of A.

Part 2: Deduplication and Semantic Vector Search Pipeline

---

1. How the System Works (The Methodology)

I implemented a two-stage pipeline to solve the challenge of tracking evolving topics across overlapping time windows and searching them based on user intent without relying on expensive LLM operations.

- **Data Ingestion and Heuristic Deduplication:** Because the three data runs have overlapping 30-day windows, the exact same topics appear multiple times. I utilized a Jaccard Similarity (Intersection over Union) approach to consolidate these without violating the rule against using an LLM for direct comparison.
    - **Tokenization:** I extracted the *title* and *summary*, stripping punctuation to create a set of raw text tokens for each valid topic (excluding the -1 outliers).
    - **Comparison:** As new topics are ingested, their token sets are compared against the pool of already processed topics. If the Jaccard score exceeds 0.5, the topic is flagged as a duplicate and discarded, leaving only a clean array of unique "master Topics".
- **Asymmetric Semantic Embedding and Retrieval:** Users query the system using natural language that explains both what they want and why. Standard keyword search struggles with underlying intent.
    - **String Formulation:** I concatenated the *title, summary,* and the primary variations of the *key_entities* into a single, information-dense string.
    - **Vectorization:** I embedded these master strings using the sentence transformer.
    - **Execution:** When a user submits a query, it is embedded using the exact same model. The system calculates the **Cosine Similarity** between the query vector and the pre-computed topic vectors, instantly returning the top k most semantically aligned results.

2. Justifications: Why I chose This Method

- **Scalability and Cost Constraint:** The assignment strictly forbids using an LLM to compare topics against each other or to check every topic against a user query.
    - The Jaccard similarity calculation is a lightweight statistical heuristic that processes deduplication locally in fractions of a second.
    - Vectorizing the topics allows for a single cosine similarity matrix multiplication during search, dropping the retrieval time to O(1) or O(logN) (with an ANN index), making the system capable of scaling to millions of topics.
- **Addressing the Asymmetric Search Problem:** I specifically chose an asymmetric retrieval model over standard semantic similarity models. The user's input is a short, conversational query, while the target document is a dense paragraph of titles and summaries. Asymmetric models are fine-tuned explicitly for this exact imbalance, which naturally boosts the cosine similarity scores for relevant matches.

3. Validation Strategy: Proving the system works

I would implement a two-pronged evaluation to validate the accuracy of this retrieval system, especially given the nuanced intent of the queries.

- **Quantitative Baseline (Information Retrieval Metrics):** I will construct a "Golden Dataset" of 50-100 manually crafted, complex user queries, explicitly mapped to the specific *topic_id* that represents the perfect answer.
    - **Recall@K (e.g., Recall@5):** I will measure how often the correct target topic appear within the top 5 results returned by the cosine similarity search. This proves the core retrieval engine works.
    - **Mean Reciprocal Rank (MRR):** I will track the exact (1st, 2nd, 3rd) of the first genuinely relevant results to ensure the mathematical distances are effectively pushing the best answers to the top.
- **Qualitative Evaluation (LLM-as-judge):** I will deploy a "Judge LLM" script, because the user's "why" (intent) is subjective, raw mathematical metrics only tell half the story.
    - I will pass the original user query and the retrieval topic's *title* and *summary* to the Judge LLM.
    - The LLM will be prompted to score the retrieval on a 1-5 scale based strictly on two criteria: **Top Relevance** and **Intent Satisfaction.**
    - Averaging these automated judge scores across the test dataset provides a scalable proxy for human satisfaction.